

# Fuzzy Logic and Semantic Web Languages

Umberto Straccia

ISTI-CNR, Pisa, Italy

<http://www.straccia.info>  
[straccia@isti.cnr.it](mailto:straccia@isti.cnr.it)

## Basics on Semantic Web Languages

RDFS, OWL, RIF

## Fuzzy RDFS Basics

Crisp RDFS

Fuzzy RDFS

## Fuzzy OWL Basics

Crisp OWL

Fuzzy OWL

## Fuzzy RIF Basics

Crisp RIF

## Conclusions

# Semantic Web Languages Basics

# Semantic Web Initiative:

## From a Knowledge Representation and Reasoning point of view

- ▶ Standards to represent Knowledge/Information at various level of granularity
- ▶ Semantics of these languages is well defined, based on some logical language
- ▶ Several tools have been developed in terms of Editors, Reasoners, . . .
- ▶ The computational complexity analysis and the development of optimised/customised algorithms becomes very important

# The Semantic Web Family of Languages

- ▶ Wide variety of languages for “Explicit Specification”
  - ▶ Graphical notations
    - ▶ Semantic networks
    - ▶ UML
    - ▶ RDF/RDFS
  - ▶ Logic based
    - ▶ Description Logics (e.g., OIL, DAML+OIL, OWL, OWL-DL, OWL-Lite, OWL 2, OWL 2 EL, OWL 2 QL, OWL 2 RL)
    - ▶ Rules (e.g., RuleML, RIF, SWRL)
    - ▶ First Order Logic (e.g., KIF)
- ▶ RDF and OWL-DL are the major players (so far ...)
- ▶ The new OWL 2 family: OWL 2, OWL 2 EL, OWL 2 QL, OWL 2 RL
- ▶ RIF (Rule interchange Format) family: RIF-Core, RIF-BLD, RIF-PRD

- ▶ **RDFS**: the triple language

$\langle \textit{subject}, \textit{predicate}, \textit{object} \rangle$

e.g.  $\langle \textit{umberto}, \textit{born}, \textit{zurich} \rangle$

- ▶ **OWL** family: the object oriented language

```
class    PERSON partial
        restriction (hasName someValuesFrom String)
        restriction (hasBirthPlace someValuesFrom GEOPLACE)
        ...
```

- ▶ **RIF** family: the rule language

forall ?Buyer ?Item ?Seller

buy(?Buyer ?Item ?Seller) : – sell(?Seller ?Item ?Buyer)

Important point: RDFS, OWL 2 and RIF are logical languages

- ▶ RDFS: logic with intensional semantics
- ▶ OWL 2: relates to the Description Logics family
- ▶ RIF: relates to the Logic Programming paradigm
- ▶ OWL 2 and RIF have extensional semantics
- ▶ We will address them from a logical point of view

# Fuzzy RDFS Basics



# Crisp RDFS Syntax

- ▶ Pairwise disjoint alphabets
  - ▶ **U** (RDFS URI references)
  - ▶ **B** (Blank nodes)
  - ▶ **L** (Literals)
- ▶ For simplicity we will denote unions of these sets simply concatenating their names
- ▶ We call elements in **UBL terms** (denoted  $t$ )
- ▶ We call elements in **B variables** (denoted  $x$ )

- ▶ **RDFS triple** (or **RDFS atom**):

$$(s, p, o) \in \mathbf{UBL} \times \mathbf{U} \times \mathbf{UBL}$$

- ▶  $s$  is the **subject**
  - ▶  $p$  is the **predicate**
  - ▶  $o$  is the **object**
- ▶ Example:

*(airplane, has, enginefault)*

## $\rho$ df (restricted RDFS)

- ▶  $\rho$ df (read rho-df, the  $\rho$  from restricted rdfs)
- ▶  $\rho$ df is defined as the following subset of the RDFS vocabulary:

$$\rho\text{df} = \{\text{sp}, \text{sc}, \text{type}, \text{dom}, \text{range}\}$$

- ▶  $(p, \text{sp}, q)$ 
  - ▶ property  $p$  is a *sub property* of property  $q$
- ▶  $(c, \text{sc}, d)$ 
  - ▶ class  $c$  is a *sub class* of class  $d$
- ▶  $(a, \text{type}, b)$ 
  - ▶  $a$  is of *type*  $b$
- ▶  $(p, \text{dom}, c)$ 
  - ▶ *domain* of property  $p$  is  $c$
- ▶  $(p, \text{range}, c)$ 
  - ▶ *range* of property  $p$  is  $c$

# RDF Semantics

- ▶ **RDF interpretation**  $\mathcal{I}$  over a vocabulary  $V$  is a tuple

$$\mathcal{I} = \langle \Delta_R, \Delta_P, \Delta_C, \Delta_L, P[\![\cdot]\!], C[\![\cdot]\!], \cdot^{\mathcal{I}} \rangle ,$$

where

- ▶  $\Delta_R, \Delta_P, \Delta_C, \Delta_L$  are the interpretations domains of  $\mathcal{I}$
- ▶  $P[\![\cdot]\!], C[\![\cdot]\!], \cdot^{\mathcal{I}}$  are the interpretation functions of  $\mathcal{I}$

$$\mathcal{I} = \langle \Delta_R, \Delta_P, \Delta_C, \Delta_L, P[\cdot], C[\cdot], \cdot^{\mathcal{I}} \rangle$$

1.  $\Delta_R$  is a nonempty set of resources, called the domain or universe of  $\mathcal{I}$ ;
2.  $\Delta_P$  is a set of property names (not necessarily disjoint from  $\Delta_R$ );
3.  $\Delta_C \subseteq \Delta_R$  is a distinguished subset of  $\Delta_R$  identifying if a resource denotes a class of resources;
4.  $\Delta_L \subseteq \Delta_R$ , the set of literal values,  $\Delta_L$  contains all plain literals in  $\mathbf{L} \cap V$ ;
5.  $P[\cdot]$  maps each property name  $p \in \Delta_P$  into a subset  $P[p] \subseteq \Delta_R \times \Delta_R$ , i.e. assigns an extension to each property name;
6.  $C[\cdot]$  maps each class  $c \in \Delta_C$  into a subset  $C[c] \subseteq \Delta_R$ , i.e. assigns a set of resources to every resource denoting a class;
7.  $\cdot^{\mathcal{I}}$  maps each  $t \in \mathbf{UL} \cap V$  into a value  $t^{\mathcal{I}} \in \Delta_R \cup \Delta_P$ , i.e. assigns a resource or a property name to each element of  $\mathbf{UL}$  in  $V$ , and such that  $\cdot^{\mathcal{I}}$  is the identity for plain literals and assigns an element in  $\Delta_R$  to elements in  $\mathbf{L}$ ;
8.  $\cdot^{\mathcal{I}}$  maps each variable  $x \in \mathbf{B}$  into a value  $x^{\mathcal{I}} \in \Delta_R$ , i.e. assigns a resource to each variable in  $\mathbf{B}$ .

# Models

Intuitively,

- ▶ A ground triple  $(s, p, o)$  in an RDF graph  $G$  will be true under the interpretation  $\mathcal{I}$  if
  - ▶  $p$  is interpreted as a property name
  - ▶  $s$  and  $o$  are interpreted as resources
  - ▶ the interpretation of the pair  $(s, o)$  belongs to the extension of the property assigned to  $p$
- ▶ Blank nodes, i.e. variables, work as existential variables: a triple  $((x, p, o)$  with  $x \in \mathbf{B}$  would be true under  $\mathcal{I}$  if
  - ▶ there exists a resource  $s$  such that  $(s, p, o)$  is true under  $\mathcal{I}$

# Models (cont.)

Let  $G$  be a graph over  $\rho$ df.

- ▶ An interpretation  $\mathcal{I}$  is a **model** of  $G$  under  $\rho$ df, denoted  $\mathcal{I} \models G$ , iff
  - ▶  $\mathcal{I}$  is an interpretation over the vocabulary  $\rho\text{df} \cup \text{universe}(G)$
  - ▶  $\mathcal{I}$  satisfies the following conditions:

Simple:

1. for each  $(s, p, o) \in G$ ,  $p^{\mathcal{I}} \in \Delta_P$  and  $(s^{\mathcal{I}}, o^{\mathcal{I}}) \in P[p^{\mathcal{I}}]$ ;

Subproperty:

1.  $P[\text{sp}^{\mathcal{I}}]$  is transitive over  $\Delta_P$ ;
2. if  $(p, q) \in P[\text{sp}^{\mathcal{I}}]$  then  $p, q \in \Delta_P$  and  $P[p] \subseteq P[q]$ ;

# Models (cont.)

Subclass:

1.  $P[\text{sc}^{\mathcal{I}}]$  is transitive over  $\Delta_C$ ;
2. if  $(c, d) \in P[\text{sc}^{\mathcal{I}}]$  then  $c, d \in \Delta_C$  and  $C[c] \subseteq C[d]$ ;

Typing I:

1.  $x \in C[c]$  iff  $(x, c) \in P[\text{type}^{\mathcal{I}}]$ ;
2. if  $(p, c) \in P[\text{dom}^{\mathcal{I}}]$  and  $(x, y) \in P[p]$  then  $x \in C[c]$ ;
3. if  $(p, c) \in P[\text{range}^{\mathcal{I}}]$  and  $(x, y) \in P[p]$  then  $y \in C[c]$ ;

Typing II:

1. For each  $e \in \text{pdf}$ ,  $e^{\mathcal{I}} \in \Delta_P$
2. if  $(p, c) \in P[\text{dom}^{\mathcal{I}}]$  then  $p \in \Delta_P$  and  $c \in \Delta_C$
3. if  $(p, c) \in P[\text{range}^{\mathcal{I}}]$  then  $p \in \Delta_P$  and  $c \in \Delta_C$
4. if  $(x, c) \in P[\text{type}^{\mathcal{I}}]$  then  $c \in \Delta_C$



# Entailment

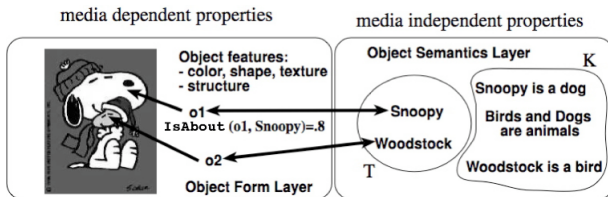
- ▶  $G$  **entails**  $H$  under  $\rho$ df, denoted  $G \models H$ , iff
  - ▶ every model under  $\rho$ df of  $G$  is also a model under  $\rho$ df of  $H$
- ▶ **Note:** often  $P[\![\text{sp}^I]\!]$  (resp.  $C[\![\text{sc}^I]\!]$ ) is also *reflexive* over  $\Delta_P$  (resp.  $\Delta_C$ )
  - ▶ We omit this requirement and, thus, do NOT support inferences such as

$$G \models (a, \text{sp}, a)$$

$$G \models (a, \text{sc}, a)$$

which anyway are of marginal interest

# Example



$$G = \left\{ \begin{array}{ll} (o1, \text{IsAbout}, \text{snoopy}) & (o2, \text{IsAbout}, \text{woodstock}) \\ (\text{snoopy}, \text{type}, \text{dog}) & (\text{woodstock}, \text{type}, \text{bird}) \\ (\text{dog}, \text{sc}, \text{animal}) & (\text{bird}, \text{sc}, \text{animal}) \end{array} \right\}$$

# Deduction System for RDF

## 1. Simple:

$$\frac{G}{G'} \text{ for } G' \subseteq G$$

## 2. Subproperty:

$$(a) \quad \frac{(A, \text{sp}, B), (B, \text{sp}, C)}{(A, \text{sp}, C)} \quad (b) \quad \frac{(A, \text{sp}, B), (X, A, Y)}{(X, B, Y)}$$

## 3. Subclass:

$$(a) \quad \frac{(A, \text{sc}, B), (B, \text{sc}, C)}{(A, \text{sc}, C)} \quad (b) \quad \frac{(A, \text{sc}, B), (X, \text{type}, A)}{(X, \text{type}, B)}$$

## 4. Typing:

$$(a) \quad \frac{(A, \text{dom}, B), (X, A, Y)}{(X, \text{type}, B)} \quad (b) \quad \frac{(A, \text{range}, B), (X, A, Y)}{(Y, \text{type}, B)}$$

## 5. Implicit Typing:

$$(a) \quad \frac{(A, \text{dom}, B), (C, \text{sp}, A), (X, C, Y)}{(X, \text{type}, B)} \quad (b) \quad \frac{(A, \text{range}, B), (C, \text{sp}, A), (X, C, Y)}{(Y, \text{type}, B)}$$

# RDFS Query Answering

- ▶ We assume that a RDF graph  $G$  is *ground* and *closed*, i.e.,  $G$  is closed under the application of the rules (2)-(5)
- ▶ **Conjunctive query**: is a Datalog-like rule of the form

$$q(\mathbf{x}) \leftarrow \exists \mathbf{y}. \tau_1, \dots, \tau_n$$

where

- ▶  $n \geq 1$ ,  $\tau_1, \dots, \tau_n$  are triples
  - ▶  $\mathbf{x}$  is a vector of variables occurring in  $\tau_1, \dots, \tau_n$ , called the *distinguished variables*
  - ▶  $\mathbf{y}$  are so-called *non-distinguished variables* and are distinct from the variables in  $\mathbf{x}$
  - ▶ each variable occurring in  $\tau_i$  is either a distinguished variable or a non-distinguished variable
- ▶ If clear from the context, we may omit the existential quantification  $\exists \mathbf{y}$
  - ▶ For instance, the query

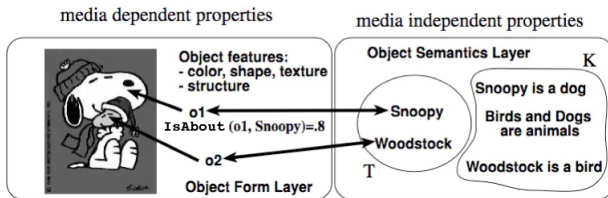
$$q(x, y) \leftarrow (x, \text{creates}, y), (x, \text{type}, \text{Flemish}), (x, \text{paints}, y), (y, \text{exhibited}, \text{Uffizi})$$

has intended meaning to retrieve all the artifacts  $x$  created by Flemish artists  $y$ , being exhibited at Uffizi Gallery

# RDF Query Answering (cont.)

- ▶ A simple query answering procedure is the following:
  - ▶ Compute the closure of a graph off-line
  - ▶ Store the RDF triples into a Relational database
  - ▶ Translate the query into a SQL statement
  - ▶ Execute the SQL statement over the relational database
- ▶ In practice, some care should be in place due to the large size of data:  $\geq 10^9$  triples
- ▶ To date, several systems exists

# Example



$$G = \left\{ \begin{array}{ll} (o1, \text{IsAbout}, \text{snoopy}) & (o2, \text{IsAbout}, \text{woodstock}) \\ (\text{snoopy}, \text{type}, \text{dog}) & (\text{woodstock}, \text{type}, \text{bird}) \\ (\text{dog}, \text{sc}, \text{animal}) & (\text{bird}, \text{sc}, \text{animal}) \end{array} \right\}$$

Consider the query

$$q(x) \leftarrow (x, \text{IsAbout}, y), (y, \text{type}, \text{Animal})$$

Then

$$\text{answer}(G, q) = \{o1, o2\}$$

# Fuzzy RDFS

- ▶ Triples may have attached a degree in  $[0, 1]$ : for  $n \in [0, 1]$

*(subject, predicate, object) : n*

- ▶ Meaning: the degree of truth of the statement is at least  $n$
- ▶ For instance,

*(o1, IsAbout, snoopy) : 0.8*

# Fuzzy RDF Syntax

- ▶ Fuzzy RDF triple (or Fuzzy RDF atom):

$$\tau: n \in (\mathbf{UBL} \times \mathbf{U} \times \mathbf{UBL}) \times [0, 1]$$

- ▶  $s \in \mathbf{UBL}$  is the **subject**
  - ▶  $p \in \mathbf{U}$  is the **predicate**
  - ▶  $o \in \mathbf{UBL}$  is the **object**
  - ▶  $n \in (0, 1]$  is the **degree of truth**
- ▶ Example:  
(audiTT, type, SportCar): 0.8



# Fuzzy RDF Semantics

- ▶ Fix a t-norm  $\otimes$
- ▶ **Fuzzy RDF interpretation**  $\mathcal{I}$  over a vocabulary  $V$  is a tuple

$$\mathcal{I} = \langle \Delta_R, \Delta_P, \Delta_C, \Delta_L, P[\![\cdot]\!], C[\![\cdot]\!], \cdot^{\mathcal{I}} \rangle ,$$

where

- ▶  $\Delta_R, \Delta_P, \Delta_C, \Delta_L$  are the interpretations domains of  $\mathcal{I}$
- ▶  $P[\![\cdot]\!], C[\![\cdot]\!], \cdot^{\mathcal{I}}$  are the interpretation functions of  $\mathcal{I}$

$$\mathcal{I} = \langle \Delta_R, \Delta_P, \Delta_C, \Delta_L, P[\cdot], C[\cdot], \cdot^{\mathcal{I}} \rangle$$

1.  $\Delta_R$  is a nonempty set of resources, called the domain or universe of  $\mathcal{I}$ ;
2.  $\Delta_P$  is a set of property names (not necessarily disjoint from  $\Delta_R$ );
3.  $\Delta_C \subseteq \Delta_R$  is a distinguished subset of  $\Delta_R$  identifying if a resource denotes a class of resources;
4.  $\Delta_L \subseteq \Delta_R$ , the set of literal values,  $\Delta_L$  contains all plain literals in  $\mathbf{L} \cap V$ ;
5.  $P[\cdot]$  maps each property name  $p \in \Delta_P$  into a function  $P[p] : \Delta_R \times \Delta_R \rightarrow [0, 1]$ , i.e. assigns a degree to each pair of resources, denoting the degree of being the pair an instance of the property  $p$ ;
6.  $C[\cdot]$  maps each class  $c \in \Delta_C$  into a function  $C[c] : \Delta_R \rightarrow [0, 1]$ , i.e. assigns a degree to every resource, denoting the degree of being the resource an instance of the class  $c$ ;
7.  $\cdot^{\mathcal{I}}$  maps each  $t \in \mathbf{UL} \cap V$  into a value  $t^{\mathcal{I}} \in \Delta_R \cup \Delta_P$ , i.e. assigns a resource or a property name to each element of  $\mathbf{UL}$  in  $V$ , and such that  $\cdot^{\mathcal{I}}$  is the identity for plain literals and assigns an element in  $\Delta_R$  to elements in  $\mathbf{L}$ ;
8.  $\cdot^{\mathcal{I}}$  maps each variable  $x \in \mathbf{B}$  into a value  $x^{\mathcal{I}} \in \Delta_R$ , i.e. assigns a resource to each variable in  $\mathbf{B}$ .

# Models

Let  $G$  be a graph over  $\rho$ df.

- ▶ An interpretation  $\mathcal{I}$  is a **model** of  $G$  under  $\rho$ df, denoted  $\mathcal{I} \models G$ , iff
  - ▶  $\mathcal{I}$  is an interpretation over the vocabulary  $\rho\text{df} \cup \text{universe}(G)$
  - ▶  $\mathcal{I}$  satisfies the following conditions:

Simple:

1. for each  $(s, p, o): n \in G, p^{\mathcal{I}} \in \Delta_P$  and  $P[\![p^{\mathcal{I}}]\!](s^{\mathcal{I}}, o^{\mathcal{I}}) \geq n$ ;

Subproperty:

1.  $P[\![sp^{\mathcal{I}}]\!](p, q) \otimes P[\![sp^{\mathcal{I}}]\!](q, r) \leq P[\![sp^{\mathcal{I}}]\!](p, r)$ ;
2.  $P[\![p^{\mathcal{I}}]\!](x, y) \otimes P[\![sp^{\mathcal{I}}]\!](p, q) \leq P[\![q^{\mathcal{I}}]\!](x, y)$ ;

# Models (cont.)

Subclass:

1.  $P[\text{sc}^I](c, d) \otimes P[\text{sc}^I](d, e) \leq P[\text{sc}^I](c, e);$
2.  $C[c^I](x) \otimes P[\text{sc}^I](c, d) \leq P[d^I](x);$

Typing I:

1.  $C[c](x) = P[\text{type}^I](x, c);$
2.  $P[\text{dom}^I](p, c) \otimes P[p](x, y) \leq C[c](x);$
3.  $P[\text{range}^I](p, c) \otimes P[p](x, y) \leq C[c](y);$

Typing II:

1. For each  $e \in \rho\text{df}$ ,  $e^I \in \Delta_P$ ;
2.  $P[\text{sp}^I](p, q)$  is defined only for  $p, q \in \Delta_P$ ;
3.  $C[\text{sc}^I](c, d)$  is defined only for  $c, d \in \Delta_C$ ;
4.  $P[\text{dom}^I](p, c)$  is defined only for  $p \in \Delta_P$  and  $c \in \Delta_C$ ;
5.  $P[\text{range}^I](p, c)$  is defined only for  $p \in \Delta_P$  and  $c \in \Delta_C$ ;
6.  $P[\text{type}^I](s, c)$  is defined only for  $c \in \Delta_C$ .

# Example & Model

$$G = \{(audiTT, type, SportsCar) : 0.8, (SportsCar, sc, PassengerCar) : 0.9\}$$

t-norm: Product

$$\mathcal{I} = \langle \Delta_R, \Delta_P, \Delta_C, \Delta_L, P[\cdot], C[\cdot], \cdot^{\mathcal{I}} \rangle$$

$$\Delta_R = \{audiTT, SportsCar, PassengerCar\}$$

$$\Delta_P = \{type, sc\}$$

$$\Delta_C = \{SportsCar, PassengerCar\}$$

$$P[type] = \{\langle audiTT, SportsCar \rangle : 0.8, \langle audiTT, PassengerCar \rangle : 0.72\}$$

$$P[sc] = \{\langle SportsCar, PassengerCar \rangle : 0.9\}$$

$$C[SportsCar] = \{audiTT : 0.8\}$$

$$C[PassengerCar] = \{audiTT : 0.72\}$$

$$t^{\mathcal{I}} = t \text{ for all } t \in \mathbf{UL}$$

$$\mathcal{I} \models G$$

$\mathcal{I}$  is a model of  $G$

# Deduction System for fuzzy RDFS

- Very simple:

$$(AG) \quad \frac{\tau_1 : n_1, \dots, \tau_k : n_k, \{\tau_1, \dots, \tau_k\} \vdash_{\text{RDFS}} \tau}{\tau : \bigotimes_i \lambda_i}$$

# Deduction System for fuzzy RDFS

1. Simple:

$$\frac{G}{G'} \text{ for } G' \subseteq G$$

2. Subproperty:

$$(a) \quad \frac{(A, \text{sp}, B) : n, (B, \text{sp}, C) : m}{(A, \text{sp}, C) : n \otimes m} \quad (b) \quad \frac{(A, \text{sp}, B) : n, (X, A, Y) : m}{(X, B, Y) : n \otimes m}$$

3. Subclass:

$$(a) \quad \frac{(A, \text{sc}, B) : n, (B, \text{sc}, C) : m}{(A, \text{sc}, C) : n \otimes m} \quad (b) \quad \frac{(A, \text{sc}, B) : n, (X, \text{type}, A) : m}{(X, \text{type}, B) : n \otimes m}$$

4. Typing:

$$(a) \quad \frac{(A, \text{dom}, B) : n, (X, A, Y) : m}{(X, \text{type}, B) : n \otimes m} \quad (b) \quad \frac{(A, \text{range}, B) : n, (X, A, Y) : m}{(Y, \text{type}, B) : n \otimes m}$$

5. Implicit Typing:

$$(a) \quad \frac{(A, \text{dom}, B) : n, (C, \text{sp}, A) : m, (X, C, Y) : r}{(X, \text{type}, B) : n \otimes m \otimes r}$$
$$(b) \quad \frac{(A, \text{range}, B) : n, (C, \text{sp}, A) : m, (X, C, Y) : r}{(Y, \text{type}, B) : n \otimes m \otimes r}$$

# Fuzzy RDFS Query Answering

- ▶ We assume that a fuzzy RDF graph  $G$  is *ground* and *closed*, i.e.,  $G$  is closed under the application of the rules (2)-(5)
- ▶ **Conjunctive query**: extends a crisp RDF query and is of the form

$$q(\mathbf{x}): s \leftarrow \exists \mathbf{y}. \tau_1: s_1, \dots, \tau_n: s_n, s = f(s_1, \dots, s_n, p_1(\mathbf{z}_1), \dots, p_h(\mathbf{z}_h))$$

where additionally

- ▶  $\mathbf{z}_i$  are tuples of terms in **UL** or variables in  $\mathbf{x}$  or  $\mathbf{y}$ ;
  - ▶  $p_j$  is an  $n_j$ -ary *fuzzy predicate* assigning to each  $n_j$ -ary tuple  $\mathbf{t}_j$  in **UL** a *score*  $p_j(\mathbf{t}_j) \in [0, 1]$ . Such predicates are called *expensive predicates* as the score is not pre-computed off-line, but is computed on query execution. We require that an  $n$ -ary fuzzy predicate  $p$  is *safe*, that is, there is not an  $m$ -ary fuzzy predicate  $p'$  such that  $m < n$  and  $p = p'$ . Informally, all parameters are needed in the definition of  $p$ ;
  - ▶  $f$  is a *scoring function*  $f: ([0, 1])^{n+h} \rightarrow [0, 1]$ , which combines the scores  $s_i$  of the  $n$  triples and the  $h$  fuzzy predicates into an overall *score* to be assigned to the rule head. We assume that  $f$  is *monotone*, that is, for each  $\mathbf{v}, \mathbf{v}' \in ([0, 1])^{n+h}$  such that  $\mathbf{v} \leq \mathbf{v}'$ , it holds  $f(\mathbf{v}) \leq f(\mathbf{v}')$ , where  $(v_1, \dots, v_{n+h}) \leq (v'_1, \dots, v'_{n+h})$  iff  $v_i \leq v'_i$  for all  $i$ ;
  - ▶ the scoring variables  $s$  and  $s_i$  are distinct from those in  $\mathbf{x}$  and  $\mathbf{y}$  and  $s$  is distinct from each  $s_i$
- ▶ If clear from the context, we may omit the existential quantification  $\exists \mathbf{y}$
  - ▶ We may omit  $s_i$  and in that case  $s_i = 1$  is assumed
  - ▶  $s = f(s_1, \dots, s_n, p_1(\mathbf{z}_1), \dots, p_h(\mathbf{z}_h))$  is called the *scoring atom*. We may also omit the scoring atom and in that case  $s = 1$  is assumed.
  - ▶ For instance, the query

$$q(x): s \leftarrow (x, \text{type}, \text{SportCar}): s_1, (x, \text{hasPrice}, y), s = s_1 \cdot \text{cheap}(y)$$

where e.g.  $\text{cheap}(p) = \text{Is}(0, 10000, 12000)$ , has intended meaning to retrieve all cheap sports car. Any answer is scored according to the product of being cheap and a sports car



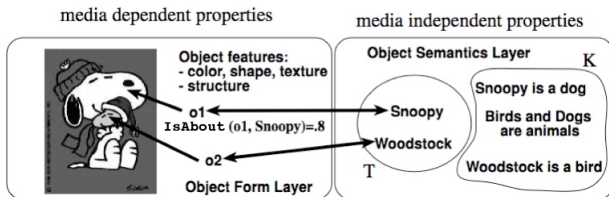
# Fuzzy RDF Query Answering (cont.)

**Top-k Retrieval:** Given a fuzzy graph  $G$ , and a query  $q$ , retrieve  $k$  answers  $\mathbf{t}$ :  $s$  with maximal scores and rank them in decreasing order relative to the score  $s$ , denoted

$$ans_k(G, q) = \text{Top}_k \text{ answer}(G, q)$$

- ▶ A simple query answering procedure is the following:
  - ▶ Compute the closure of a graph off-line
  - ▶ Store the fuzzy RDF triples into a relational database supporting Top-k retrieval (e.g., RankSQL, Postgres)
  - ▶ Translate the fuzzy query into a top-k SQL statement
  - ▶ Execute the SQL statement over the relational database
  - ▶ Few systems exists, e.g. FuzzyRDF, AnQL (<http://anql.deri.org/>)

# Example



$$G = \left\{ \begin{array}{ll} (o1, IsAbout, snoopy): 0.8 & (o2, IsAbout, woodstock): 0.9 \\ (snoopy, type, dog) & (woodstock, type, bird) \\ (Bird, sc, SmallAnimal): 0.7 & (Dog, sc, SmallAnimal): 0.4 \\ (dog, sc, Animal) & (bird, sc, Animal) \\ (SmallAnimal, sc, Animal) & \end{array} \right\}$$

Consider the query

$$q(x): s \leftarrow (x, IsAbout, y): s_1, (y, type, Animal): s_2, s = s_1 \cdot s_2$$

Then (under any t-norm)

$$ans(G, q) = \{o1: 0.32, o2: 0.63\}, \quad ans_1(G, q) = \{o2: 0.63\}$$

# Fuzzy OWL Basics

# OWL

- ▶ OWL Family
  - ▶ **OWL full** is union of OWL syntax and RDF (Undecidable)
  - ▶ **OWL DL** restricted to Description Logics fragment (decidable in NEXPTIME)
  - ▶ **OWL Lite** is “easier to implement” subset of OWL DL (decidable in EXPTIME)
  - ▶ **OWL 2** new OWL standard (decidable, NEXPTIME-hard)
  - ▶ OWL 2 profiles
    - ▶ OWL QL (query answering LOGSPACE)
    - ▶ OWL EL (classification in polynomial time)
    - ▶ OWL RL (EXPTIME, intersection of Description Logics and Logic Programming)
- ▶ Semantic layering
  - ▶ OWL 2 within **Description Logic (DL) fragment**
- ▶ OWL DL is based on ***SHOIN(D)***
- ▶ OWL Lite is based on ***SHIF(D)***
- ▶ OWL 2 is based on ***SROIQ(D)***
- ▶ OWL QL is based on ***DL – Lite***
- ▶ OWL EL is based on ***EL***
- ▶ OWL RL is a **Datalog** fragment

# The DL Family

- ▶ A given DL is defined by set of concept and role forming operators
- ▶ Basic language:  $\mathcal{ALC}$  (Attributive Language with Complement)

| Syntax             | Semantics                          | Example                                                                                          |
|--------------------|------------------------------------|--------------------------------------------------------------------------------------------------|
| $C, D \rightarrow$ | $\top$                             |                                                                                                  |
|                    | $\perp$                            |                                                                                                  |
|                    | $A$                                | <i>Human</i>                                                                                     |
|                    | $C \sqcap D$                       | <i>Human</i> $\sqcap$ <i>Male</i>                                                                |
|                    | $C \sqcup D$                       | <i>Nice</i> $\sqcup$ <i>Rich</i>                                                                 |
|                    | $\neg C$                           | $\neg$ <i>Meat</i>                                                                               |
|                    | $\exists R.C$                      | $\exists$ <i>has_child</i> . <i>Blond</i>                                                        |
|                    | $\forall R.C$                      | $\forall$ <i>has_child</i> . <i>Human</i>                                                        |
| $C \sqsubseteq D$  | $\forall x. C(x) \Rightarrow D(x)$ | <i>Happy_Father</i> $\sqsubseteq$ <i>Man</i> $\sqcap$ $\exists$ <i>has_child</i> . <i>Female</i> |
| $a:C$              | $C(a)$                             | <i>John:Happy_Father</i>                                                                         |

# Note on DL Naming

$\mathcal{AL}$ :  $C, D \longrightarrow \top \mid \perp \mid A \mid C \sqcap D \mid \neg A \mid \exists R.T \mid \forall R.C$

$\mathcal{C}$ : Concept negation,  $\neg C$ . Thus,  $\mathcal{ALC} = \mathcal{AL} + \mathcal{C}$

$\mathcal{S}$ : Used for  $\mathcal{ALC}$  with transitive roles  $\mathcal{R}_+$

$\mathcal{U}$ : Concept disjunction,  $C_1 \sqcup C_2$

$\mathcal{E}$ : Existential quantification,  $\exists R.C$

$\mathcal{H}$ : Role inclusion axioms,  $R_1 \sqsubseteq R_2$ , e.g. *is\_component\_of*  $\sqsubseteq$  *is\_part\_of*

$\mathcal{N}$ : Number restrictions,  $(\geq n R)$  and  $(\leq n R)$ , e.g.  $(\geq 3 \text{ has\_Child})$  (has at least 3 children)

$\mathcal{Q}$ : Qualified number restrictions,  $(\geq n R.C)$  and  $(\leq n R.C)$ , e.g.  $(\leq 2 \text{ has\_Child.Adult})$  (has at most 2 adult children)

$\mathcal{O}$ : Nominals (singleton class),  $\{a\}$ , e.g.  $\exists \text{has\_child}.\{mary\}$ .

**Note:**  $a:C$  equiv to  $\{a\} \sqsubseteq C$  and  $(a, b):R$  equiv to  $\{a\} \sqsubseteq \exists R.\{b\}$

$\mathcal{I}$ : Inverse role,  $R^-$ , e.g. *isPartOf* = *hasPart*<sup>-</sup>

$\mathcal{F}$ : Functional role,  $f$ , e.g. *functional*(*hasAge*)

$\mathcal{R}_+$ : transitive role, e.g. *transitive*(*isPartOf*)

For instance,

|         |     |                                                                                    |          |
|---------|-----|------------------------------------------------------------------------------------|----------|
| $SHIF$  | $=$ | $S + \mathcal{H} + \mathcal{I} + \mathcal{F} = \mathcal{ALCR}_+HIF$                | OWL-Lite |
| $SHOIN$ | $=$ | $S + \mathcal{H} + \mathcal{O} + \mathcal{I} + \mathcal{N} = \mathcal{ALCR}_+HOIN$ | OWL-DL   |
| $SROIQ$ | $=$ | $S + \mathcal{R} + \mathcal{O} + \mathcal{I} + \mathcal{Q} = \mathcal{ALCR}_+ROIN$ | OWL 2    |

# Semantics of Additional Constructs

- $\mathcal{H}$ : Role inclusion axioms,  $\mathcal{I} \models R_1 \sqsubseteq R_2$  iff  $R_1^{\mathcal{I}} \subseteq R_2^{\mathcal{I}}$
- $\mathcal{N}$ : Number restrictions,  
 $(\geq n R)^{\mathcal{I}} = \{x \in \Delta^{\mathcal{I}} : |\{y \mid \langle x, y \rangle \in R^{\mathcal{I}}\}| \geq n\},$   
 $(\leq n R)^{\mathcal{I}} = \{x \in \Delta^{\mathcal{I}} : |\{y \mid \langle x, y \rangle \in R^{\mathcal{I}}\}| \leq n\}$
- $\mathcal{Q}$ : Qualified number restrictions,  
 $(\geq n R.C)^{\mathcal{I}} = \{x \in \Delta^{\mathcal{I}} : |\{y \mid \langle x, y \rangle \in R^{\mathcal{I}} \wedge y \in C^{\mathcal{I}}\}| \geq n\},$   
 $(\leq n R.C)^{\mathcal{I}} = \{x \in \Delta^{\mathcal{I}} : |\{y \mid \langle x, y \rangle \in R^{\mathcal{I}} \wedge y \in C^{\mathcal{I}}\}| \leq n\}$
- $\mathcal{O}$ : Nominals (singleton class),  $\{a\}^{\mathcal{I}} = \{a^{\mathcal{I}}\}$
- $\mathcal{I}$ : Inverse role,  $(R^{-})^{\mathcal{I}} = \{\langle x, y \rangle \mid \langle y, x \rangle \in R^{\mathcal{I}}\}$
- $\mathcal{F}$ : Functional role,  $\mathcal{I} \models \text{fun}(f)$  iff  $\forall z \forall y \forall z$  if  $\langle x, y \rangle \in f^{\mathcal{I}}$  and  $\langle x, z \rangle \in f^{\mathcal{I}}$  the  $y = z$
- $\mathcal{R}_+$ : transitive role,  
 $(R_+)^{\mathcal{I}} = \{\langle x, y \rangle \mid \exists z \text{ such that } \langle x, z \rangle \in R^{\mathcal{I}} \wedge \langle z, y \rangle \in R^{\mathcal{I}}\}$

# Basics on Concrete Domains

- ▶ **Concrete domains**: reals, integers, strings, ...

*(tim, 14):hasAge*

*(sf, "SoftComputing"):hasAcronym*

*(source1, "ComputerScience"):isAbout*

*(service2, "InformationRetrievalTool"):Matches*

*Minor = Person*  $\sqcap \exists hasAge. \leq_{18}$

- ▶ Semantics: a clean separation between “object” classes and concrete domains
  - ▶  $D = \langle \Delta_D, \Phi_D \rangle$
  - ▶  $\Delta_D$  is an interpretation domain
  - ▶  $\Phi_D$  is the set of concrete domain predicates  $d$  with a predefined arity  $n$  and **fixed** interpretation  $d^D \subseteq \Delta_D^n$
  - ▶ Concrete properties:  $R^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}} \times \Delta_D$
- ▶ Notation:  $(D)$ . E.g.,  $\mathcal{ALC}(D)$  is  $\mathcal{ALC}$  + concrete domains



# DL Knowledge Base

- ▶ A DL **Knowledge Base** is a pair  $KB = \langle \mathcal{T}, \mathcal{A} \rangle$ , where
  - ▶  $\mathcal{T}$  is a **TBox**
    - ▶ containing general inclusion axioms of the form  $C \sqsubseteq D$ ,
    - ▶ concept definitions of the form  $A = C$
    - ▶ primitive concept definitions of the form  $A \sqsubseteq C$
    - ▶ role inclusions of the form  $R \sqsubseteq P$
    - ▶ role equivalence of the form  $R = P$
  - ▶  $\mathcal{A}$  is a **ABox**
    - ▶ containing assertions of the form  $a:C$
    - ▶ containing assertions of the form  $(a, b):R$
    - ▶ containing (in) equality Axioms of the form  $a = b$  and  $a \neq b$
- ▶ An interpretation  $\mathcal{I}$  is a model of  $KB$ , written  $\mathcal{I} \models KB$  iff  $\mathcal{I} \models \mathcal{T}$  and  $\mathcal{I} \models \mathcal{A}$ , where
  - ▶  $\mathcal{I} \models \mathcal{T}$  ( $\mathcal{I}$  is a model of  $\mathcal{T}$ ) iff  $\mathcal{I}$  is a model of each element in  $\mathcal{T}$
  - ▶  $\mathcal{I} \models \mathcal{A}$  ( $\mathcal{I}$  is a model of  $\mathcal{A}$ ) iff  $\mathcal{I}$  is a model of each element in  $\mathcal{A}$

# Basic Inference Problems (Formally)

**Consistency:** Check if knowledge is meaningful

- ▶ Is  $KB$  satisfiability?  $\mapsto$  Is there some model  $\mathcal{I}$  of  $KB$  ?
- ▶ Is  $C$  satisfiability?  $\mapsto C^{\mathcal{I}} \neq \emptyset$  for some some model  $\mathcal{I}$  of  $KB$  ?

**Subsumption:** structure knowledge, compute taxonomy

- ▶  $KB \models C \sqsubseteq D$  ?  $\mapsto$  Is it true that  $C^{\mathcal{I}} \subseteq D^{\mathcal{I}}$  for all models  $\mathcal{I}$  of  $KB$  ?

**Equivalence:** check if two classes denote same set of instances

- ▶  $KB \models C = D$  ?  $\mapsto$  Is it true that  $C^{\mathcal{I}} = D^{\mathcal{I}}$  for all models  $\mathcal{I}$  of  $KB$  ?

**Instantiation:** check if individual  $a$  instance of class  $C$

- ▶  $KB \models a:C$  ?  $\mapsto$  Is it true that  $a^{\mathcal{I}} \in C^{\mathcal{I}}$  for all models  $\mathcal{I}$  of  $KB$  ?

**Retrieval:** retrieve set of individuals that instantiate  $C$

- ▶ Compute the set  $\{a \mid KB \models a:C\}$

# Reduction to Satisfiability

Problems are all **reducible** to KB satisfiability

**Subsumption:**  $KB \models C \sqsubseteq D$  iff  $\langle \mathcal{T}, \mathcal{A} \cup \{a:C \sqcap \neg D\} \rangle$  not satisfiable, where  $a$  is a new individual

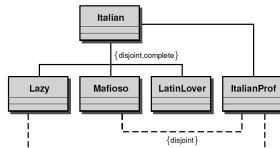
**Equivalence:**  $KB \models C = D$  iff  $KB \models C \sqsubseteq D$  and  $KB \models D \sqsubseteq C$

**Instantiation:**  $KB \models a:C$  iff  $\langle \mathcal{T}, \mathcal{A} \cup \{a:\neg C\} \rangle$  not satisfiable

**Retrieval:** The computation of the set  $\{a \mid KB \models a:C\}$  is reducible to the instance checking problem

# Exercise

**Problem:** Consider the following conceptual schema



Encode it into Description logics and prove that  $KB \models \text{ItalianProf} \sqsubseteq \text{LatinLover}$

**Solution:**

- $\text{Lazy} \sqsubseteq \text{Italian}$
- $\text{Mafioso} \sqsubseteq \text{Italian}$
- $\text{LatinLover} \sqsubseteq \text{Italian}$
- $\text{Italian} \sqsubseteq (\text{Lazy} \sqcup \text{Mafioso} \sqcup \text{LatinLover})$
- $\text{ItalianProf} \sqsubseteq \text{Italian}$
- $\text{Lazy} \sqsubseteq \neg \text{Mafioso}$
- $\text{Lazy} \sqsubseteq \neg \text{LatinLover}$
- $\text{Mafioso} \sqsubseteq \neg \text{LatinLover}$
- $\text{Mafioso} \sqsubseteq \neg \text{ItalianProf}$
- $\text{Lazy} \sqsubseteq \neg \text{ItalianProf}$

# Reasoning in DLs: Basics

- ▶ Tableaux algorithm deciding satisfiability
- ▶ Try to build a **tree-like model**  $\mathcal{I}$  of the KB
- ▶ Decompose concepts  $C$  syntactically
  - ▶ Apply tableau **expansion rules**
  - ▶ Infer constraints on elements of model
- ▶ Tableau rules correspond to constructors in logic ( $\sqcap, \sqcup, \dots$ )
  - ▶ Some rules are **nondeterministic** (e.g.,  $\sqcup, \leq$ )
  - ▶ In practice, this means **search**
- ▶ Stop when no more rules applicable or **clash** occurs
  - ▶ Clash is an obvious contradiction, e.g.,  $A(x), \neg A(x)$
- ▶ Cycle check (**blocking**) may be needed for termination

# Negation Normal Form (NNF)

- We have to transform concepts into **Negation Normal Form**: push negation inside using de Morgan' laws

$$\begin{aligned}\neg \top &\mapsto \perp \\ \neg \perp &\mapsto \top \\ \neg \neg C &\mapsto C \\ \neg (C_1 \sqcap C_2) &\mapsto \neg C_1 \sqcup \neg C_2 \\ \neg (C_1 \sqcup C_2) &\mapsto \neg C_1 \sqcap \neg C_2\end{aligned}$$

and

$$\begin{aligned}\neg (\exists R.C) &\mapsto \forall R. \neg C \\ \neg (\forall R.C) &\mapsto \exists R. \neg C\end{aligned}$$

# Completion-Forest

- ▶ This is a forest of trees, where
  - ▶ each node  $x$  is labelled with a set  $\mathcal{L}(x)$  of concepts
  - ▶ each edge  $\langle x, y \rangle$  is labelled with  $\mathcal{L}(\langle x, y \rangle) = \{R\}$  for some role  $R$  (edges correspond to relationships between pairs of individuals)
- ▶ The forest is initialized with
  - ▶ a root node  $a$ , labelled  $\mathcal{L}(a) = \emptyset$  for each individual  $a$  occurring in the KB
  - ▶ an edge  $\langle a, b \rangle$  labelled  $\mathcal{L}(\langle a, b \rangle) = \{R\}$  for each  $(a, b):R$  occurring in the KB
- ▶ Then, for each  $a:C$  occurring in the KB, set  $\mathcal{L}(a) \rightarrow \mathcal{L}(a) \cup \{C\}$
- ▶ The algorithm expands the tree either by extending  $\mathcal{L}(x)$  for some node  $x$  or by adding new leaf nodes.
- ▶ Edges are added when expanding  $\exists R.C$
- ▶ A completion-forest contains a **clash** if, for a node  $x$ ,  $\{C, \neg C\} \subseteq \mathcal{L}(x)$
- ▶ If nodes  $x$  and  $y$  are connected by an edge  $\langle x, y \rangle$ , then  $y$  is called a successor of  $x$  and  $x$  is called a predecessor of  $y$ . Ancestor is the transitive closure of predecessor.
- ▶ A node  $y$  is called an  $R$ -successor of a node  $x$  if  $y$  is a successor of  $x$  and  $\mathcal{L}(\langle x, y \rangle) = \{R\}$ .
- ▶ The algorithm returns “satisfiable” if rules can be applied s.t. they yield a clash-free, complete (no more rules can be applied) completion forest

# $\mathcal{ALC}$ Tableau rules without GCI's

| Rule        | Description                                                                                                                                                                                                                                                                                                            |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $(\sqcap)$  | <p>if 1. <math>C_1 \sqcap C_2 \in \mathcal{L}(x)</math> and<br/>2. <math>\{C_1, C_2\} \not\subseteq \mathcal{L}(x)</math><br/>then <math>\mathcal{L}(x) \rightarrow \mathcal{L}(x) \cup \{C_1, C_2\}</math></p>                                                                                                        |
| $(\sqcup)$  | <p>if 1. <math>C_1 \sqcup C_2 \in \mathcal{L}(x)</math> and<br/>2. <math>\{C_1, C_2\} \cap \mathcal{L}(x) = \emptyset</math><br/>then <math>\mathcal{L}(x) \rightarrow \mathcal{L}(x) \cup \{C\}</math> for some <math>C \in \{C_1, C_2\}</math></p>                                                                   |
| $(\exists)$ | <p>if 1. <math>\exists R.C \in \mathcal{L}(x)</math> and<br/>2. <math>x</math> has no <math>R</math>-successor <math>y</math> with <math>C \in \mathcal{L}(y)</math><br/>then create a new node <math>y</math> with <math>\mathcal{L}(\langle x, y \rangle) = \{R\}</math> and <math>\mathcal{L}(y) = \{C\}</math></p> |
| $(\forall)$ | <p>if 1. <math>\forall R.C \in \mathcal{L}(x)</math> and<br/>2. <math>x</math> has an <math>R</math>-successor <math>y</math> with <math>C \notin \mathcal{L}(y)</math><br/>then <math>\mathcal{L}(y) \rightarrow \mathcal{L}(y) \cup \{C\}</math></p>                                                                 |



# Fuzzy DLs

The semantics is an immediate consequence of the First-Order-Logic translation of DLs expressions

Interpretation:

|                   |   |                                                                       |               |   |             |
|-------------------|---|-----------------------------------------------------------------------|---------------|---|-------------|
| $\mathcal{I}$     | = | $\Delta^{\mathcal{I}}$                                                | $\otimes$     | = | t-norm      |
| $C^{\mathcal{I}}$ | : | $\Delta^{\mathcal{I}} \rightarrow [0, 1]$                             | $\oplus$      | = | s-norm      |
| $R^{\mathcal{I}}$ | : | $\Delta^{\mathcal{I}} \times \Delta^{\mathcal{I}} \rightarrow [0, 1]$ | $\neg$        | = | negation    |
|                   |   |                                                                       | $\Rightarrow$ | = | implication |

|           | Syntax                      | Semantics                                                                                                                 |
|-----------|-----------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Concepts: | $C, D \longrightarrow \top$ | $\top^{\mathcal{I}}(x) = 1$                                                                                               |
|           | $\perp$                     | $\perp^{\mathcal{I}}(x) = 0$                                                                                              |
|           | $A$                         | $A^{\mathcal{I}}(x) \in [0, 1]$                                                                                           |
|           | $C \sqcap D$                | $(C_1 \sqcap C_2)^{\mathcal{I}}(x) = C_1^{\mathcal{I}}(x) \otimes C_2^{\mathcal{I}}(x)$                                   |
|           | $C \sqcup D$                | $(C_1 \sqcup C_2)^{\mathcal{I}}(x) = C_1^{\mathcal{I}}(x) \oplus C_2^{\mathcal{I}}(x)$                                    |
|           | $\neg C$                    | $(\neg C)^{\mathcal{I}}(x) = \neg C^{\mathcal{I}}(x)$                                                                     |
|           | $\exists R.C$               | $(\exists R.C)^{\mathcal{I}}(x) = \sup_{y \in \Delta^{\mathcal{I}}} R^{\mathcal{I}}(x, y) \otimes C^{\mathcal{I}}(y)$     |
|           | $\forall R.C$               | $(\forall R.C)^{\mathcal{I}}(x) = \inf_{y \in \Delta^{\mathcal{I}}} R^{\mathcal{I}}(x, y) \Rightarrow C^{\mathcal{I}}(y)$ |

Assertions:  $a:C: r, \mathcal{I} \models a:C: r$  iff  $C^{\mathcal{I}}(a^{\mathcal{I}}) \geq r$  (similarly for roles)

► individual  $a$  is instance of concept  $C$  at least to degree  $r, r \in [0, 1] \cap \mathbb{Q}$

Inclusion axioms:  $C \sqsubseteq D: r,$

►  $\mathcal{I} \models C \sqsubseteq D: r$  iff  $\inf_{x \in \Delta^{\mathcal{I}}} C^{\mathcal{I}}(x) \Rightarrow D^{\mathcal{I}}(x) \geq r$

# Main Inference Problems

**Graded entailment:** Check if DL axiom  $\alpha$  is entailed to degree at least  $r$

►  $KB \models \alpha : r ?$

**BED:** Best Entailment Degree problem

►  $bed(KB, \alpha) = \sup\{r \mid KB \models \alpha : r\}$

**BSD:** Best Satisfiability Degree problem

►  $bsd(KB, C) = \sup_{\mathcal{I} \models KB} \{C^{\mathcal{I}}(a^{\mathcal{I}})\}$ , for new individual  $a$

**Top-k retrieval:** Retrieve the top-k individuals that instantiate  $C$  w.r.t. best truth value bound

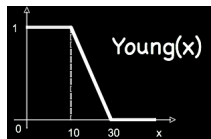
►  $ans_k(KB, C) = Top_k\{\langle a, r \rangle \mid r = bed(KB, a:C)\}$

# Towards fuzzy OWL 2 and its Profiles

- ▶ Recall that OWL 2 relates to  $\mathcal{SROIQ}(D)$
- ▶ We need to extend the semantics of fuzzy  $\mathcal{ALC}$  to fuzzy  $\mathcal{SROIQ}(D)$
- ▶ Additionally, we add
  - ▶ **modifiers** (e.g., *very*)
  - ▶ **concrete fuzzy concepts** (e.g., *Young*)
  - ▶ both additions have **explicit** membership functions

# Concrete fuzzy concepts

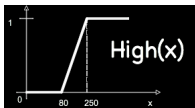
- ▶ E.g., *Small*, *Young*, *High*, etc. with **explicit** membership function
- ▶ Use the idea of concrete domains:
  - ▶  $D = \langle \Delta_\beta, \Phi_\beta \rangle$
  - ▶  $\Delta_\beta$  is an interpretation domain
  - ▶  $\Phi_\beta$  is the set of concrete fuzzy domain predicates  $d$  with a predefined arity  $n = 1, 2$  and **fixed** interpretation  $d^\beta : \Delta_\beta^n \rightarrow [0, 1]$
  - ▶ For instance,



$$\begin{aligned} \text{Minor} &= \text{Person} \sqcap \exists \text{hasAge.} \leq 18 \\ \text{YoungPerson} &= \text{Person} \sqcap \exists \text{hasAge. Young} \\ &\quad \text{functional}(\text{hasAge}) \end{aligned}$$

# Modifiers

- ▶ *Very, moreOrLess, slightly*, etc.
- ▶ Apply to fuzzy sets to change their membership function
  - ▶  $very(x) = x^2$
  - ▶  $slightly(x) = \sqrt{x}$
- ▶ For instance,



$$SportsCar = Car \sqcap \exists speed.very(High)$$

# Fuzzy *SHOIN*(D)

Concepts:

|                        | Syntax                 | Semantics                                                                                                         |
|------------------------|------------------------|-------------------------------------------------------------------------------------------------------------------|
| $C, D \longrightarrow$ | $\top$                 | $\top(x)$                                                                                                         |
|                        | $\perp$                | $\perp(x)$                                                                                                        |
|                        | $A$                    | $A(x)$                                                                                                            |
|                        | $(C \sqcap D)$         | $C_1(x) \otimes C_2(x)$                                                                                           |
|                        | $(C \sqcup D)$         | $C_1(x) \oplus C_2(x)$                                                                                            |
|                        | $(\neg C)$             | $\neg C(x)$                                                                                                       |
|                        | $(\exists R.C)$        | $\exists x R(x, y) \otimes C(y)$                                                                                  |
|                        | $(\forall R.C)$        | $\forall x R(x, y) \Rightarrow C(y)$                                                                              |
|                        | $\{a\}$                | $x = a$                                                                                                           |
|                        | $(\geq n R)$           | $\exists y_1, \dots, y_n. \otimes_{i=1}^n R(x, y_i) \otimes \otimes_{1 \leq i < j \leq n} y_i \neq y_j$           |
|                        | $(\leq n R)$           | $\forall y_1, \dots, y_{n+1}. \otimes_{i=1}^{n+1} R(x, y_i) \Rightarrow \oplus_{1 \leq i < j \leq n+1} y_i = y_j$ |
|                        | $FCC$                  | $\mu_{FCC}(x)$                                                                                                    |
|                        | $M(C)$                 | $\mu_M(C(x))$                                                                                                     |
|                        | $\sum_i w_i \cdot C_i$ | $w_1 \cdot C_1(x) + \dots + w_n \cdot C_n(x) \quad (\sum_i w_i = 1)$                                              |
| $R \longrightarrow$    | $P$                    | $P(x, y)$                                                                                                         |
|                        | $P^-$                  | $P(y, x)$                                                                                                         |

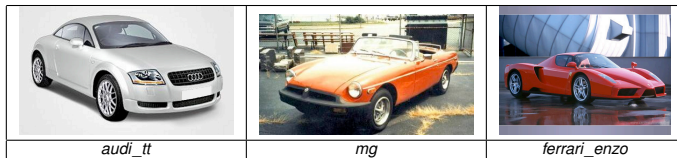
Assertions:

|                          | Syntax        | Semantics        |
|--------------------------|---------------|------------------|
| $\alpha \longrightarrow$ | $a:C: r$      | $C(a) \geq r$    |
|                          | $(a, b):R: r$ | $R(a, b) \geq r$ |

Axioms:

|                        | Syntax               | Semantics                                                                |
|------------------------|----------------------|--------------------------------------------------------------------------|
| $\tau \longrightarrow$ | $C \sqsubseteq D: r$ | $\forall x. C(x) \Rightarrow D(x) \geq r$                                |
|                        | $fun(R)$             | $\forall x \forall y \forall z R(x, y) \wedge R(x, z) \Rightarrow y = z$ |
|                        | $trans(R)$           | $(\exists z R(x, z) \wedge R(z, y)) \Rightarrow R(x, y)$                 |

# Example (Graded Entailment)

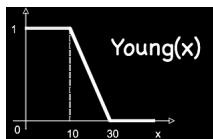


| <i>Car</i>          | <i>speed</i> |
|---------------------|--------------|
| <i>audi_tt</i>      | 243          |
| <i>mg</i>           | $< 170$      |
| <i>ferrari_enzo</i> | $\geq 350$   |

*SportsCar* = *Car*  $\sqcap \exists \text{hasSpeed}.\text{very(High)}$

*KB*  $\models$  *ferrari\_enzo*:*SportsCar* : 1  
*KB*  $\models$  *audi\_tt*:*SportsCar* : 0.92  
*KB*  $\models$  *mg*: $\neg$ *SportsCar* : 0.72

# Example (Graded Subsumption)



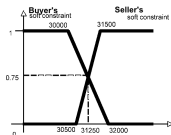
$$\begin{aligned} Minor &= Person \sqcap \exists hasAge. \leq_{18} \\ YoungPerson &= Person \sqcap \exists hasAge. Young \end{aligned}$$

$$KB \models Minor \sqsubseteq YoungPerson: 0.6$$

Note: without an explicit membership function of *Young*, this inference cannot be drawn



# Example (Simplified Negotiation)



- ▶ a car seller sells an Audi TT for 31500 €, as from the catalog price.
- ▶ a buyer is looking for a sports-car, but wants to pay not more than around 30000 €
- ▶ classical DLs: the problem relies on the crisp conditions on price
- ▶ more fine grained approach: to consider prices as fuzzy sets (as usual in negotiation)
  - ▶ seller may consider optimal to sell above 31500 €, but can go down to 30500 €
  - ▶ the buyer prefers to spend less than 30000 €, but can go up to 32000 €
    - $AudiTT = SportsCar \sqcap \exists hasPrice.R(x; 30500, 31500)$
    - $Query = SportsCar \sqcap \exists hasPrice.L(x; 30000, 32000)$
  - ▶ highest degree to which the concept  $C = AudiTT \sqcap Query$  is satisfiable is 0.75 (the possibility that the Audi TT and the query **matches** is 0.75)
  - ▶ the car may be sold at 31250 €

# Reasoning in Fuzzy $\mathcal{ALC}$ , under Zadeh Semantics

- ▶ Applies technique based on Mixed Integer Programming (MILP) for fuzzy propositional logic to  $\mathcal{ALC}$  calculus
- ▶ For each concept assertion  $\alpha$  of the form  $a:C$ , we use variable  $x_\alpha$ , which holds the degree of truth of  $\alpha$
- ▶ It can be shown that

$$\begin{aligned}bed(KB, (a, b):R) &= bed(KB \cup \{b:B: 1\}, a:\exists R.B) \\bed(KB, C \sqsubseteq D) &= \min x \text{ such that } KB \cup \{b:C \sqcap \neg D: 1 - x\} \text{ satisfiable} \\bed(KB, a:C) &= \min x \text{ such that } KB \cup \{a:\neg C: 1 - x\} \text{ satisfiable} \\bsd(KB, C) &= \min -x \text{ such that } KB \cup \{b:C: x\} \text{ satisfiable}\end{aligned}$$

where  $b$  is a new individual and  $B$  is a new concept

# Satisfiability Testing

- ▶ The notion of **completion forest**  $\mathcal{F}$  is similar to the case of  $\mathcal{ALC}$ 
  - ▶  $\mathcal{F}$  contains a root node  $a_i$  for each individual  $a_i$  occurring in  $\mathcal{A}$
  - ▶  $\mathcal{F}$  contains an edge  $\langle a, b \rangle$  for each  $(a, b):R: n \in \mathcal{A}$
  - ▶ for each  $a:C: n \in \mathcal{A}$ , we add both  $C$  to  $\mathcal{L}(a)$  and  $x_{a:C} \geq n$  to  $\mathcal{C}_{\mathcal{F}}$
  - ▶ for each  $(a, b):R: n \in \mathcal{A}$ , we add both  $R$  to  $\mathcal{L}(\langle a, b \rangle)$  and  $x_{(a, b):R} \geq n$  to  $\mathcal{C}_{\mathcal{F}}$
- ▶ The notion of blocking is as for crisp  $\mathcal{ALC}$
- ▶  $\mathcal{F}$  is then expanded by repeatedly applying the rules described below
- ▶ The completion-forest is complete when none of the rules are applicable
- ▶ Then, the bMILP problem on  $\mathcal{C}_{\mathcal{F}}$  is solved

# Fuzzy $\mathcal{ALC}$ Tableau rules with GCI's (Zadeh semantics)

| Rule              | Description                                                                                                                                                                                                                                                      |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| (var)             | For variable $x_{v:C}$ add $x_{v:C} \in [0, 1]$ to $\mathcal{C}_{\mathcal{F}}$ . For variable $x_{(v,w):R}$ , add $x_{(v,w):R} \in [0, 1]$ to $\mathcal{C}_{\mathcal{F}}$                                                                                        |
| ( $\bar{A}$ )     | if $\neg A \in \mathcal{L}(v)$ then add $x_{v:A} = 1 - x_{v:\neg A}$ to $\mathcal{C}_{\mathcal{F}}$                                                                                                                                                              |
| ( $\perp$ )       | If $\perp \in \mathcal{L}(v)$ then add $x_{v:\perp} = 0$ to $\mathcal{C}_{\mathcal{F}}$                                                                                                                                                                          |
| ( $\top$ )        | If $\top \in \mathcal{L}(v)$ then add $x_{v:\top} = 1$ to $\mathcal{C}_{\mathcal{F}}$                                                                                                                                                                            |
| ( $\sqcap$ )      | if $C_1 \sqcap C_2 \in \mathcal{L}(v)$ , $v$ is not indirectly blocked<br>then $\mathcal{L}(v) \rightarrow \mathcal{L}(v) \cup \{C_1, C_2\}$ , and add $x_{v:C_1} \otimes x_{v:C_2} \geq x_{v:C_1 \sqcap C_2}$ to $\mathcal{C}_{\mathcal{F}}$                    |
| ( $\sqcup$ )      | if $C_1 \sqcup C_2 \in \mathcal{L}(v)$ , $v$ is not indirectly blocked<br>then $\mathcal{L}(v) \rightarrow \mathcal{L}(v) \cup \{C_1, C_2\}$ , and add $x_{v:C_1} \oplus x_{v:C_2} \geq x_{v:C_1 \sqcup C_2}$ to $\mathcal{C}_{\mathcal{F}}$                     |
| ( $\forall$ )     | if $\forall R.C \in \mathcal{L}(v)$ , $v$ is not indirectly blocked<br>then $\mathcal{L}(w) \rightarrow \mathcal{L}(w) \cup \{C\}$ , and add $x_{w:C} \geq x_{v:\forall R.C} \otimes x_{(v,w):R}$ to $\mathcal{C}_{\mathcal{F}}$                                 |
| ( $\exists$ )     | if $\exists R.C \in \mathcal{L}(v)$ , $v$ is not blocked<br>then create new node $w$ with $\mathcal{L}(\langle v, w \rangle) = \{R\}$ and $\mathcal{L}(w) = \{C\}$ , and add $x_{w:C} \otimes x_{(v,w):R} \geq x_{v:\exists R.C}$ to $\mathcal{C}_{\mathcal{F}}$ |
| ( $\sqsubseteq$ ) | if $C \sqsubseteq D$ : $n \in \mathcal{T}$ , $v$ is not indirectly blocked<br>then $\mathcal{L}(v) \rightarrow \mathcal{L}(v) \cup \{C, D\}$ , and add $x_{v:D} \geq x_{v:C} \otimes n$ to $\mathcal{C}_{\mathcal{F}}$                                           |

- ▶ As fuzzy DL extensions involving

- ▶ concrete domains
  - ▶ modifiers

are combinations of linear functions, they can be translated into MILP equations

- ▶ Therefore, the algorithm can be extended to fuzzy DLs with concrete domains
- ▶ For a fuzzy DL system, see fuzzyDL, Fire, DeLorean
- ▶ Systems supporting top-k retrieval: based on DL-Lite/DLR-Lite
  - ▶ DLMedia (Ontology mediated Multimedia data retrieval)
  - ▶ Softfacts (Ontology mediated Database data retrieval)
- ▶ A proposal to encode Fuzzy OWL 2 using OWL 2 exists
- ▶ Protege plug-in to encode Fuzzy OWL ontologies using OWL 2 Editor Protege exists

## Fuzzy RIF Basics

# Crisp RIF

- ▶ RIF is a family of rule languages
  - ▶ **RIF-Core**: this corresponds to Datalog
  - ▶ **RIF-BLD, Basic Logic Dialect**: this corresponds to definite Horn rules with equality and a standard first-order semantics, and subsumes RIF-Core
  - ▶ **RIF-PRD, Production Rule Dialect**: subsumes RIF-Core.
    - ▶ Production rules have an if part, or condition, and a then part, or action. The “condition” is as usual, but the “then” part contains actions. An action can assert facts, modify facts, retract facts, and have other side-effects

# LPs Basics (for ease, Datalog)

- ▶ **Predicates** are  $n$ -ary
- ▶ **Terms** are variables or constants
- ▶ **Facts** ground atoms

For instance,

*has\_parent(mary, jo)*

- ▶ **Rules** are of the form

$$P(\mathbf{x}) \leftarrow \varphi(\mathbf{x}, \mathbf{y})$$

where  $\varphi(\mathbf{x}, \mathbf{y})$  is a formula built from atoms of the form  $B(\mathbf{z})$   
and connectors  $\wedge, \vee, 0, 1$

For instance,

$$has\_father(x, y) \leftarrow has\_parent(x, y) \wedge Male(y)$$



- ▶ **Extensional database** (EDB): set of facts
- ▶ **Intentional database** (IDB): set of rules
- ▶ **Logic Program**  $\mathcal{P}$ :
  - ▶  $\mathcal{P} = EDB \cup IDB$
  - ▶ No predicate symbol in  $EDB$  occurs in the head of a rule in  $IDB$ 
    - ▶ The principle is that we do not allow that  $IDB$  may redefine the extension of predicates in  $EDB$
- ▶  $EDB$  is usually, stored into a relational database

# LPs Semantics: FOL semantics

►  $\mathcal{P}^*$  is constructed as follows:

1. set  $\mathcal{P}^*$  to the set of all ground instantiations of rules in  $\mathcal{P}$ ;
2. replace a fact  $p(\mathbf{c})$  in  $\mathcal{P}^*$  with the rule  $p(\mathbf{c}) \leftarrow 1$
3. if atom  $A$  is not head of any rule in  $\mathcal{P}^*$ , then add  $A \leftarrow 0$  to  $\mathcal{P}^*$ ;
4. replace several rules in  $\mathcal{P}^*$  having same head

$$\left. \begin{array}{l} A \leftarrow \varphi_1 \\ A \leftarrow \varphi_2 \\ \vdots \\ A \leftarrow \varphi_n \end{array} \right\} \text{ with } A \leftarrow \varphi_1 \vee \varphi_2 \vee \dots \vee \varphi_n .$$

- Note: in  $\mathcal{P}^*$  each atom  $A \in B_{\mathcal{P}}$  is head of **exactly one** rule
- **Herbrand Base** of  $\mathcal{P}$  is the set  $B_{\mathcal{P}}$  of ground atoms
- **Interpretation** is a function  $I : B_{\mathcal{P}} \rightarrow \{0, 1\}$ .
- **Model**  $I \models \mathcal{P}$  iff for all  $r \in \mathcal{P}^*$   $I \models r$ , where  $I \models A \leftarrow \varphi$  iff  $I(\varphi) \leq I(A)$

- ▶ **Entailment**: for a ground atom  $p(\mathbf{c})$

$$\mathcal{P} \models p(\mathbf{c}) \text{ iff all models of } \mathcal{P} \text{ satisfy } p(\mathbf{c})$$

- ▶ **Least model**  $M_{\mathcal{P}}$  of  $\mathcal{P}$  exists and is **least fixed-point** of

$$T_{\mathcal{P}}(I)(A) = I(\varphi), \text{ for all } A \leftarrow \varphi \in \mathcal{P}^*$$

- ▶  $M$  can be computed as the limit of

$$\begin{aligned} \mathbf{l}_0 &= \mathbf{0} \\ \mathbf{l}_{i+1} &= T_{\mathcal{P}}(\mathbf{l}_i) . \end{aligned}$$

# LP Query Answering

- ▶ **Query**: is a rule of the form

$$q(\mathbf{x}) \leftarrow \varphi(\mathbf{x}, \mathbf{y})$$

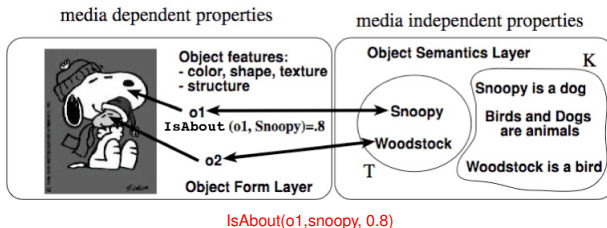
- ▶ If  $\mathcal{P} \models q(\mathbf{c})$  then  $\mathbf{c}$  is called an **answer** to  $q$
- ▶ The **answer set** of  $q$  w.r.t.  $\mathcal{P}$  is defined as

$$ans(\mathcal{P}, q) = \{\mathbf{c} \mid \mathcal{P} \models q(\mathbf{c})\}$$

- ▶ Efficient query answering algorithms exists

# Representing degrees in LPs

- ▶ How can we represent truth degrees in LPs?
- ▶ Unfortunately, no standard exists yet
- ▶ However, as simple encoding is to make transform an  $n$ -ary predicate  $P$  into an  $n + 1$ -ary predicate, where the additional argument stores the value:



- ▶ For instance, in LP systems we may write

$$q(h, s) \leftarrow \text{hasLocation}(h, hl), \text{hasLocation}(\text{train}, cl), \text{close}(hl, cl, s1), \text{cheap}(h, s2), s \text{ is } s1 \cdot s2$$

- ▶ But, then again such statements have to be appropriately managed the system according to the underlying uncertainty or vagueness theory

# Fuzzy LPs Basics

- ▶ We consider fuzzy LPs, which extends classical LPs, where

- ▶ **Truth space** is  $[0, 1]$
- ▶ **Interpretation** is a mapping  $I : B_{\mathcal{P}} \rightarrow [0, 1]$
- ▶ **Generalized LP rules** are of the form

$$R(\mathbf{x}) \leftarrow \exists \mathbf{y}. f(R_1(\mathbf{z}_1), \dots, R_k(\mathbf{z}_k), p_1(\mathbf{z}'_1), \dots, p_h(\mathbf{z}'_h))$$

- ▶ **Meaning of rules**: “take the truth-values of all  $R_i(\mathbf{z}_i), p_j(\mathbf{z}'_j)$ , combine them using the truth combination function  $f$ , and assign the result to  $R(\mathbf{x})$ ”
- ▶ **Facts**: ground expressions of the form  $R(\mathbf{c}) : n$ 
  - ▶ **Meaning of facts**: “the degree of truth of  $R(\mathbf{c})$  is at least  $n$ ”
- ▶ **Fuzzy LP**: a set of facts (extensional database) and a set of rules (intentional database). No extensional relation may occur in the head of a rule

► Rules:

$$R(\mathbf{x}) \leftarrow \exists \mathbf{y}. \varphi(\mathbf{x}, \mathbf{y})$$

1.  $\mathbf{x}$  are the *distinguished variables*;
2.  $s$  is the *score variable*, taking values in  $[0, 1]$ ;
3.  $\mathbf{y}$  are existentially quantified variables, called *non-distinguished variables*;
4.  $\varphi(\mathbf{x}, \mathbf{y})$  is  $f(\mathbf{R}(\mathbf{z}), \mathbf{p}(\mathbf{z}'))$ , where  $\mathbf{R}$  is a vector of predicates  $R_i$  and  $\mathbf{p}$  is a vector of fuzzy predicates  $p_j$ ;
5.  $\mathbf{z}, \mathbf{z}'$  are tuples of constants in  $KB$  or variables in  $\mathbf{x}$  or  $\mathbf{y}$ ;
6.  $p_j$  is an  $n_j$ -ary *fuzzy predicate* assigning to each  $n_j$ -ary tuple  $\mathbf{c}_j$  the score  $p_j(\mathbf{c}_j) \in [0, 1]$ ;
7.  $f$  is a monotone *scoring function*  $f: [0, 1]^{k+h} \rightarrow [0, 1]$ , which combines the scores of the  $h$  fuzzy predicates  $p_j(\mathbf{c}_j)$  with the  $k$  scores  $R_i(\mathbf{c}_i)$

# Semantics of fuzzy LPs

- ▶  $\mathcal{P}^*$  is constructed as follows (as for the classical case):
  1. set  $\mathcal{P}^*$  to the set of all ground instantiations of rules in  $\mathcal{P}$ ;
  2. replace a fact  $p(\mathbf{c})$  in  $\mathcal{P}^*$  with the rule  $p(\mathbf{c}) \leftarrow 1$
  3. if atom  $A$  is not head of any rule in  $\mathcal{P}^*$ , then add  $A \leftarrow 0$  to  $\mathcal{P}^*$ ;
  4. replace several rules in  $\mathcal{P}^*$  having same head

$$\left. \begin{array}{l} A \leftarrow \varphi_1 \\ A \leftarrow \varphi_2 \\ \vdots \\ A \leftarrow \varphi_n \end{array} \right\} \text{ with } A \leftarrow \varphi_1 \vee \varphi_2 \vee \dots \vee \varphi_n .$$

- ▶ Note: in  $\mathcal{P}^*$  each atom  $A \in B_{\mathcal{P}}$  is head of **exactly one** rule
- ▶ **Herbrand Base** of  $\mathcal{P}$  is the set  $B_{\mathcal{P}}$  of ground atoms
- ▶ **Interpretation** is a function  $I : B_{\mathcal{P}} \rightarrow [0, 1]$ .
- ▶ **Model**  $I \models \mathcal{P}$  iff for all  $r \in \mathcal{P}^*$   $I \models r$ , where  $I \models A \leftarrow \varphi$  iff  $I(\varphi) \leq I(A)$
- ▶ **Note:**

$$I(f(R_1(\mathbf{c}_1), \dots, R_k(\mathbf{c}_k), p_1(\mathbf{c}'_1), \dots, p_h(\mathbf{c}'_h))) = f(I(R_1(\mathbf{c}_1)), \dots, I(R_k(\mathbf{c}_k)), p_1(\mathbf{c}'_1), \dots, p_h(\mathbf{c}'_h)))$$



# Fuzzy LP Query Answering

- ▶ **Least model**  $M_{\mathcal{P}}$  of  $\mathcal{P}$  exists and is **least fixed-point** of

$$T_{\mathcal{P}}(I)(A) = I(\varphi), \text{ for all } A \leftarrow \varphi \in \mathcal{P}^*$$

- ▶  $M$  can be computed as the limit of

$$\begin{aligned} \mathbf{l}_0 &= \mathbf{0} \\ \mathbf{l}_{i+1} &= T_{\mathcal{P}}(\mathbf{l}_i) . \end{aligned}$$

- ▶ **Entailment**: for a ground expression  $q(\mathbf{c})$ :  $s$ ,  $s \in [0, 1]$

$$\mathcal{P} \models q(\mathbf{c}) : s \text{ iff least model of } \mathcal{P} \text{ satisfies } I(q(\mathbf{c})) \geq s$$

- ▶ We say that  $s$  is *tight* iff  $s = \sup\{s' \mid \mathcal{P} \models q(\mathbf{c}) : s'\}$
- ▶ If  $\mathcal{P} \models q(\mathbf{c}) : s$  and  $s$  is tight then  $\mathbf{c} : s$  is called an *answer* to  $q$
- ▶ The **answer set** of  $q$  w.r.t.  $\mathcal{P}$  is defined as

$$\text{ans}(\mathcal{P}, q) = \{\mathbf{c} : s \mid \mathcal{P} \models q(\mathbf{c}) : s, s \text{ is tight}\}$$

**Top-k Retrieval**: Given a fuzzy LP  $\mathcal{P}$ , and a query  $q$ , retrieve  $k$  answers  $\mathbf{c} : s$  with maximal scores and rank them in decreasing order relative to the score  $s$ , denoted

$$\text{ans}_k(\mathcal{P}, q) = \text{Top}_k \text{ ans}(\mathcal{P}, q) .$$

- ▶ Fuzzy LPs may be tricky:

$$\begin{array}{l} A: 0 \\ A \leftarrow (A + 1)/2 \end{array}$$

- ▶ In the minimal model the truth of  $A$  is 1 (requires infinitely many  $T_{\mathcal{P}}$  iterations)!
- ▶ There are several ways to avoid this pathological behavior:
  - ▶ We may consider  $L = \{0, \frac{1}{n}, \frac{2}{n} \dots, \frac{n-1}{n}, 1\}$ ,  $n$  natural number, e.g.  $n = 100$
  - ▶ In  $A \leftarrow f(B_1, \dots, B_n)$ ,  $f$  is bounded, i.e.  $f(x_1, \dots, x_n) \leq x_i$

# Example: Soft shopping agent

- I may represent my preferences in Logic Programming with the rules

$$Pref_1(x, p) \leftarrow HasPrice(x, p) \wedge LS(10000, 14000)(p)$$

$$Pref_2(x) \leftarrow HasKM(x, k) \wedge LS(13000, 17000)(k)$$

$$Buy(x, p) \leftarrow 0.7 \cdot Pref_1(x, p) + 0.3 \cdot Pref_2(x)$$

| ID   | MODEL      | PRICE | KM    |
|------|------------|-------|-------|
| 455  | MAZDA 3    | 12500 | 10000 |
| 34   | ALFA 156   | 12000 | 15000 |
| 1812 | FORD FOCUS | 11000 | 16000 |
| ⋮    | ⋮          | ⋮     | ⋮     |

- **Problem:** All tuples of the database have a score:
  - We cannot compute the score of all tuples, then rank them. Brute force approach not feasible for very large databases
- **Top-k problem:** Determine **efficiently** just the **top-k ranked** tuples, without evaluating the score of all tuples. E.g. top-3 tuples

| ID   | PRICE | SCORE |
|------|-------|-------|
| 1812 | 11000 | 0.6   |
| 455  | 12500 | 0.56  |
| 34   | 12000 | 0.50  |

# Top- $k$ retrieval in LPs

- ▶ If the database contains a huge amount of facts, a brute force approach fails:
  - ▶ one cannot anymore compute the score of all tuples, rank all of them and only then return the top- $k$
- ▶ Better solutions exists for restricted fuzzy LP languages:  
Datalog + restriction on the score combination functions appearing in the body

# Basic Idea

- ▶ We do not compute all answers, but determine answers incrementally
- ▶ At each step  $i$ , from the tuples seen so far in the database, we compute a **threshold**  $\delta$
- ▶ The threshold  $\delta$  has the property that any successively retrieved answer will have a score  $s \leq \delta$
- ▶ Therefore, we can **stop** as soon as we have gathered  $k$  answers **above**  $\delta$ , because any successively computed answer will have a score below  $\delta$

# Example

Logic Program  $\mathcal{P}$  is

$$\begin{aligned}q(x) &\leftarrow p(x) \\ p(x) &\leftarrow \min(r_1(x, y), r_2(y, z))\end{aligned}$$

| <i>RecordID</i> | $r_1$    |          |          | $r_2$    |          |          |
|-----------------|----------|----------|----------|----------|----------|----------|
| 1               | <i>a</i> | <i>b</i> | 1.0      | <i>m</i> | <i>h</i> | 0.95     |
| 2               | <i>c</i> | <i>d</i> | 0.9      | <i>m</i> | <i>j</i> | 0.85     |
| 3               | <i>e</i> | <i>f</i> | 0.8      | <i>f</i> | <i>k</i> | 0.75     |
| 4               | <i>l</i> | <i>m</i> | 0.7      | <i>m</i> | <i>n</i> | 0.65     |
| 5               | <i>o</i> | <i>p</i> | 0.6      | <i>p</i> | <i>q</i> | 0.55     |
| $\vdots$        | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ |

What is

$$Top_1(\mathcal{P}, q) = Top_1\{\langle c, s \rangle \mid \mathcal{P} \models q(c, s)\} \text{ ?}$$

$$q(x) \leftarrow p(x)$$

$$p(x) \leftarrow \min(r_1(x, y), r_2(y, z))$$

|   | RecordID | $r_1$ |   |     | $r_2$ |   |      |   |
|---|----------|-------|---|-----|-------|---|------|---|
|   | 1        | a     | b | 1.0 | m     | h | 0.95 |   |
|   | 2        | c     | d | 0.9 | m     | j | 0.85 |   |
|   | 3        | e     | f | 0.8 | f     | k | 0.75 | ← |
| → | 4        | l     | m | 0.7 | m     | n | 0.65 |   |
|   | 5        | o     | p | 0.6 | p     | q | 0.55 |   |
|   | ⋮        | ⋮     | ⋮ | ⋮   | ⋮     | ⋮ | ⋮    |   |
|   | ⋮        | ⋮     | ⋮ | ⋮   | ⋮     | ⋮ | ⋮    |   |

Action: **STOP**, top-1 tuple score is equal or above threshold  $0.75 = \max(\min(1.0, 0.75), \min(0.7, 0.95))$

| Queue | $\delta$ | Predicate | Answers                                           |
|-------|----------|-----------|---------------------------------------------------|
| —     | 0.75     | q         | $\langle e, 0.75 \rangle \langle l, 0.7 \rangle$  |
|       |          | p         | $\langle e, 0.75 \rangle, \langle l, 0.7 \rangle$ |

$$Top_1(\mathcal{P}, q) = \{ \langle e, 0.75 \rangle \}$$

Note: no further answer will have score above threshold  $\delta$

# Threshold computation

For an intentional predicate  $p$ , head of a rule  $r : p(\mathbf{x}) \leftarrow f(p_1, p_2, \dots, p_n)$ .

- ▶ consider a threshold variable  $\delta^p$
- ▶ with  $r.\mathbf{t}_{p_i}^\perp$  ( $r.\mathbf{t}_{p_i}^\top$ ) we denote the last tuple seen (the top ranked one) in  $\text{rankedList}(p, r)$
- ▶ we define

$$p_i^\top = \max(\delta^{p_i}, r.\mathbf{t}_{p_i}^\top.\text{score})$$

$$p_i^\perp = \delta^{p_i}$$

- ▶ if  $p_i$  is an extensional predicate, we define

$$p_i^\top = r.\mathbf{t}_{p_i}^\top.\text{score}$$

$$p_i^\perp = r.\mathbf{t}_{p_i}^\perp.\text{score}$$

- ▶ for rule  $r$  we consider the equation  $\delta(r)$

$$\delta^p = \max(f(p_1^\perp, p_2^\top, \dots, p_n^\top), f(p_1^\top, p_2^\perp, \dots, p_n^\top), \dots, f(p^\top, p^\top, \dots, p_n^\perp))$$

- ▶ consider the set of equations of all equations involving intentional predicates, i.e.

$$\Delta = \bigcup_{r \in P} \{\delta(r)\}.$$

- ▶ for a query  $q(\mathbf{x})$ , the threshold  $\delta$  of the *TopAnswers* algorithm is defined as to be

$$\delta = \bar{\delta}^q,$$

where  $\bar{\delta}^q$  is the solution to  $\delta^q$  in the minimal solution  $\bar{\Delta}$  of the set of equations  $\Delta$ .

- ▶ note that  $\bar{\delta}^q$ , can be computed iteratively as least fixed-point



# Conclusions

- ▶ Main Semantic Web Languages standards:
  - ▶ RDFS: the triple language
  - ▶ OWL family: the object oriented language
  - ▶ RIF family: the rule language
- ▶ Several working tools exists for them

- ▶ Fuzzyfication of these languages is an emerging demand along several directions such as
  1. Fuzzyfying predicates
  2. Several t-norms
  3. Allowing modifiers
  4. Explicit representation of membership functions
  5. Fuzzy quantifiers
  6. Annotating statements with truth degrees
  7. Fuzzy spatial, fuzzy time
  8. Standardisation of Syntax & Semantics
- ▶ Fuzzy RDFS & SPARQL.
  - ▶ AnQL (Annotated RDFS and SPARQL) is the most advanced framework so far
  - ▶ Can easily be implemented
  - ▶ But, no efficient system exists yet
- ▶ Fuzzy OWL 2
  - ▶ Extends OWL 2 with many “fuzzy features”
  - ▶ Reasoning procedures very tricky, decidability and computational complexity not simple to figure out
  - ▶ Some reasoners exists, fuzzyDL, Fire, DeLorean
  - ▶ Expandable Fuzzy OWL 2 standard encoding proposed and Protege plug-in available
- ▶ Fuzzy RIF
  - ▶ Fuzzy RIF can be obtained from the various Fuzzy LPs framework developed so far
  - ▶ No system exists yet supporting Fuzzy RIF
  - ▶ More problematic for large scale databases, no efficient support for top-k retrieval